

ScholarMate

사용자 맞춤 장학금 추천 플랫폼

박해준 | Frontend Developer

장학금 탐색 → 맞춤 추천 → 관심 등록 → 캘린더 관리 → 커뮤니티 공유

React, Vite, TanStack Query, Redux Toolkit, Axios 기반으로 사용자 흐름과 성능 개선을 함께 구현한 프로젝트입니다.

저는 이 프로젝트에서 프론트엔드 개발을 담당했습니다.

장학금 탐색, 맞춤 추천, 관심목록, 캘린더, 커뮤니티, 쪽지, 인증 흐름을 구현했고, 사용자가 장학금을 찾는 순간부터 지원 일정을 관리하는 과정까지 하나의 흐름으로 연결하는 데 집중했습니다.

이 프로젝트를 통해 API 연동, 인증 상태 관리, 서버 상태 캐싱, 낙관적 업데이트, 성능 최적화, 접근성 검증까지 경험했습니다.

84.6%

홈 JS chunk 감소

87.6%

홈 CSS chunk 감소

100

홈 desktop Lighthouse Performance

100

홈 mobile Lighthouse Performance

사용자 맞춤 장학금 추천 플랫폼

기간: 2025.04 - 2025.11 | 인원: 2명 (프론트엔드 1명, 백엔드 1명) | 역할: 프론트엔드

프로젝트 소개

ScholarMate는 여러 기관에 흩어진 장학금 정보를 통합 탐색하고, 사용자의 학적·지역·소득·성적 조건을 기반으로 맞춤 장학금을 추천하며, 관심 장학금의 마감 일정을 관리할 수 있도록 만든 웹 서비스입니다.

문제 정의

장학금 정보는 기관별로 흩어져 있고 모집 기간, 성적, 소득, 거주 지역, 학과 조건이 복잡합니다.
사용자가 직접 비교하고 마감일을 관리해야 하므로 지원 기회를 놓치기 쉽습니다.

사용한 기술 스택

Frontend: React, Vite, React Router
State/API: TanStack Query, Redux Toolkit, Axios
Style/UI: Tailwind CSS, Ant Design, React Calendar
Quality: Vite production build, Node test runner, axe-core, Lighthouse, Chrome Performance API, Playwright, GitHub Actions

System Architecture



사용자 흐름과 담당 구현

기능을 개별 화면이 아니라 실제 지원 준비까지 이어지는 사용자 흐름으로 연결했습니다.

01

로그인

02

정보 입력

03

맞춤 추천

04

관심 등록

05

마감 관리

06

정보 공유

인증 / 라우팅

JWT 만료 여부를 앱 진입과 경로 변경 시
검사하고, Redux Toolkit으로 로그인 여부와
인증 확인 완료 상태를 전역 관리했습니다.
'PrivateRoute'를 통해 장학금 목록, 추천,
관심목록 등 핵심 페이지 접근을 제어했습니다.

탐색 / 추천

검색어, 유형 필터, 정렬, 페이지네이션을
구현했습니다.
추천 결과의 선별 이유 JSON을 정규화해 추천
이유를 설명 가능하게 표시했습니다.

캘린더 / 커뮤니티

관심 장학금 마감일을 캘린더에 표시하고
D-day 배지를 구현했습니다.
좋아요 / 북마크에는 낙관적 업데이트와
실패 롤백을 적용했습니다.

핵심 구현

인증 흐름과 보호 라우팅, 장학금 검색/필터/정렬/페이지네이션,
추천 결과 상세보기와 관심 등록, 캘린더 D-day 배지,
커뮤니티 낙관적 업데이트와 쪽지 polling을 구현했습니다.

구조 개선

페이지 중심으로 시작한 코드를 api, features, shared로 분리했습니다.
추천, 장학금 목록, 캘린더, 커뮤니티는 컴포넌트와 유틸 함수로 나누어
page 컴포넌트가 데이터 흐름 중심의 역할을 하도록 정리했습니다.

기술 선택과 설계 판단

React와 Vite

React는 장학금 목록, 추천 결과, 캘린더, 커뮤니티처럼 상태가 많은 화면을 컴포넌트 단위로 분리하기 적합했습니다. Vite는 빠른 개발 서버와 명확한 production build 산출물을 제공해 번들 크기와 성능 개선 결과를 확인하기 좋았습니다.

TanStack Query와 Redux Toolkit 분리

서버 데이터와 클라이언트 상태의 성격이 다르다고 판단해 역할을 분리했습니다. 상태의 성격에 따라 client state와 server state를 분리했습니다.

TanStack Query

장학금 목록, 공지사항, 추천 결과, 관심 장학금처럼 서버에서 가져오는 데이터에 사용했습니다.

서버 상태에는 캐싱, stale time, refetch, mutation invalidation이 필요했기 때문입니다.

Axios interceptor

모든 API 요청에서 토큰 첨부과 인증 실패 처리를 반복하지 않기 위해 Axios instance와 interceptor를 사용했습니다.

요청 전 access token 만료 여부를 확인하고, 필요한 경우 refresh token으로 재발급을 시도하도록 구성했습니다.

Redux Toolkit

로그인 여부와 인증 확인 완료처럼 앱 전체에 공유되고, 헤더와 보호 라우트가 함께 참조하는 앱 전역 client state에 사용했습니다.

Feature 단위 구조 개선

처음에는 페이지 중심으로 구현했지만, 기능이 커지면서 반복되는 로직을 `api`, `features`, `shared`로 분리했습니다.

추천, 장학금 목록, 캘린더, 커뮤니티 목록은 컴포넌트와 유틸 함수로 나누어 page 컴포넌트가 데이터 흐름 중심의 역할을 하도록 정리했습니다.

핵심 기능 구현

사용자가 장학금을 찾고 저장하고 마감일까지 관리하는 흐름을 만들었습니다.

장학금 탐색과 관심 등록

API 응답 형태가 배열, `results`, `data` 등으로 달라질 수 있는 상황을 정규화해 UI에서 일관되게 사용할 수 있도록 처리했습니다.

상세 모달, 홈페이지 링크 정규화, 관심 장학금 등록 / 해제를 구현했습니다.

모바일에서는 테이블을 대신 카드 UI를 제공해 작은 화면에서도 핵심 정보를 빠르게 확인할 수 있게 했습니다.

맞춤 추천 흐름

로그인 토큰 기반 추천 API를 호출하고, 프로필 또는 장학 정보가 없는 경우 입력 페이지로 이동하는 복구 동선을 제공했습니다.
추천 카드 안에서 상세보기, 홈페이지 이동, 관심 등록을 처리했고,
추천 UI를 feature 단위 컴포넌트로 분리했습니다.

캘린더와 쪽지

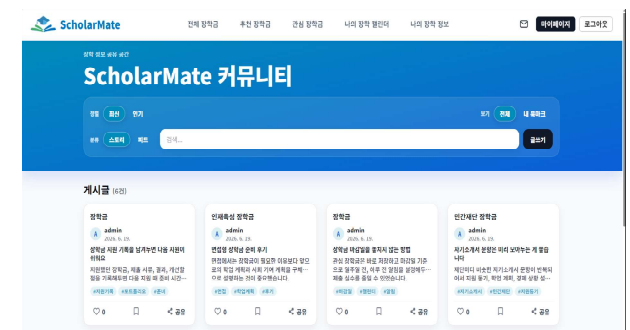
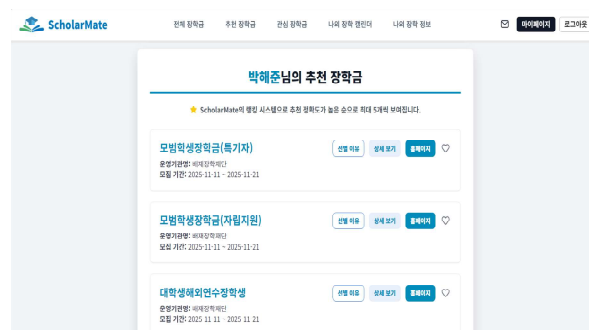
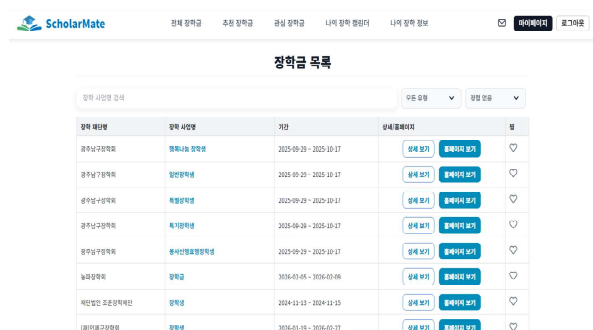
YYYY-MM-DD 문자열을 로컬 date 객체로 직접 생성해 timezone으로 날짜가 하루 밀리는 문제를 방지했습니다.

제출 완료 상태는 localStorage에 저장해 재방문 시에도 유지되도록 했습니다.

썬더는 polling 기반으로 새 메시지를 갱신하고, optimistic message와 서버 메시지를 병합해 중복 메시지가 생기지 않도록 처리했습니다.

커뮤니티

검색어, 카테고리, 정렬, 페이지 정보를 URL query와 동기화했습니다.
좋아요와 북마크는 클릭 즉시 UI에 반영하고 실패 시 이전 상태로 롤백했습니다.



Troubleshooting 1. 홈 초기 로딩 성능 최적화

측정 → 병목 확인 → 라이브러리 제거와 이미지 최적화 → 재측정

문제

홈 화면에서 여러 캐러셀 라이브러리와 원본 이미지가 함께 로드되어 초기 번들이 커지고 이미지 리소스 비중이 높았습니다.

또한, 히어로 이미지가 CSS background로 처리되어 브라우저가 이미지 우선순위를 판단하기 어려웠습니다.

해결

외부 캐러셀 라이브러리를 제거하고 React state와 CSS transition으로 히어로 슬라이더를 구현했습니다.

첫 번째 히어로 이미지에는 `loading="eager"`와 `fetchPriority="high"`를 적용하고, 하단 섹션은 `IntersectionObserver`, `React.lazy`, `Suspense`로 지연 로딩했습니다.

JPG / PNG 이미지는 WebP로 변환하고 추가 압축했습니다.

180.96kB → 27.91kB

홈 JS chunk 84.6% 감소

20.60kB → 2.55kB

홈 CSS chunk 87.6% 감소

1,356,919B → 511,565B

브라우저 총 encoded body size 약 62.3% 감소

1,154,182B → 67,318B

브라우저 이미지 encoded body size 약 94.2% 감소

80 → 100

홈 desktop Lighthouse Performance

92 → 100

홈 mobile Lighthouse Performance

production build 환경에서 Chrome Performance API로 리소스 로딩과 초기 렌더링 지표를 측정하고, 확인된 병목을 기준으로 개선했습니다.

Troubleshooting 2. 토큰 만료와 동시 refresh 요청

여러 API 요청이 동시에 실패해도 refresh 요청은 한 번만 실행되도록 처리했습니다.

Before

API A 401 → refresh
API B 401 → refresh
API C 401 → refresh

동시 요청마다 refresh API가 중복 호출될 수 있음

문제

access token이 만료된 상태에서 여러 API 요청이 동시에 발생하면 refresh 요청이 중복 실행될 수 있었습니다.

이 경우 인증 상태가 불안정해지고 일부 요청이 실패한 상태로 남을 수 있습니다.

해결

Axios request interceptor에서 access token을 자동 첨부하고, 요청 전 token 만료 여부를 검사했습니다.

만료된 경우 refresh token으로 새 access token을 발급받도록 했습니다. 이때, 여러 요청이 동시에 refresh를 시도하지 않도록 `refreshPromise`를 공유해 하나의 refresh 요청을 기다리게 했습니다.

After

API A 401 }
API B 401 } → shared refreshPromise
API C 401 }

single-flight 구조로 refresh race condition 가능성을 낮춤

결과

만료 토큰으로 인한 반복 실패를 줄였고, 동시에 여러 요청이 실패해도 refresh race condition 가능성을 낮췄습니다.

refresh 실패 시에는 토큰을 정리하고 로그인 화면으로 복구되도록 처리했습니다.

Troubleshooting 3. 데이터 정규화와 낙관적 업데이트

문제

프론트엔드 폼 필드명과 백엔드 API 필드명이 달라 추천 입력 데이터가 불안정해질 수 있었습니다.
또한, 좋아요, 북마크, 쪽지 전송처럼 즉각적인 반응이 중요한 기능에서 서버 응답을 기다린 뒤 UI를 변경하면 사용자가 느리게 느낄 수 있었습니다.

추천 입력 데이터의 일관성과 사용자 액션의 즉각적인 반응을 함께 고려했습니다.

데이터 정규화

장학 정보 입력에서는 서버 응답을 폼 상태로 변환하는 함수와 폼 상태를 서버 payload로 변환하는 함수를 분리했습니다.

빈 선택값은 `null`, GPA는 number, 체크박스는 boolean으로 보장했습니다.

낙관적 업데이트

좋아요와 북마크는 먼저 UI를 갱신하고 실패 시 이전 상태로 롤백하는 낙관적 업데이트를 적용했습니다.

쪽지 전송은 임시 ID를 가진 optimistic message를 먼저 추가하고, 서버 응답 이후 실제 메시지로 병합했습니다.

단위 테스트

변환 로직은 UI 컴포넌트에서 분리하고 pure function으로 만들어 Node test runner로 검증했습니다.

결과

추천 API에 전달되는 입력 데이터의 일관성을 높였고, 사용자 액션에 대한 체감 반응 속도를 개선했습니다.

실패 상황에서도 상태 불일치가 남지 않도록 롤백과 병합 로직을 함께 적용했습니다.

Troubleshooting 4. 쪽지 미읽음 집계와 즉시 읽음 처리

문제

실제 미읽음 쪽지는 3개였지만 헤더 배지에는 6개로 표시됐습니다.
대화 진입 후에도 서버 읽음 처리 전까지 배지가 남았습니다.

원인과 근본 수정

대화 참여자 JOIN으로 미읽음 수가 중복 집계됐습니다.
백엔드 Count에 distinct를 적용해 메시지 ID 기준으로 집계했습니다.

API 계약 정리

백엔드가 정확한 unread_count를 반환하도록 수정하고,
프론트의 participant_count 나눗셈 보정을 제거했습니다.

즉시 UI 반영

쪽지 항목 클릭 시 해당 대화의 배지를 즉시 0으로 갱신했습니다.
읽음 처리 완료 이벤트로 헤더와 목록 상태를 동기화했습니다.

자동화 검증

백엔드 API 테스트로 3개가 정확히 반환되는지 검증했습니다.
프론트 단위 테스트와 E2E로 3 → 0 흐름을 확인했습니다.

성능 및 품질 검증

실제 배포에 가까운 production build 기준으로 측정하고, 테스트와 접근성 검증을 함께 수행했습니다. production build를 만든 뒤 정적 서버로 제공하고, API 요청은 로컬 백엔드로 프록시해 측정했습니다.

416 ms

홈 desktop LCP

0.008

홈 CLS

100

Desktop Lighthouse

100

Mobile Lighthouse

측정 결과

- 홈 desktop: FCP 400 ms, LCP 416 ms, CLS 0.008, TBT 근사 174 ms, 실패 요청 0, 콘솔 에러 0
- 홈 mobile: FCP 312 ms, LCP 312 ms, CLS 0.001 미만, TBT 근사 159 ms, 실패 요청 0, 콘솔 에러 0
- 공지사항 desktop: FCP 368 ms, LCP 1.3 s, CLS 0.001 미만, TBT 근사 225 ms, 실패 요청 0, 콘솔 에러 0
- Lighthouse desktop: Performance 100, Accessibility 100, Best Practices 100, SEO 100
- Lighthouse mobile: Performance 100, Accessibility 100, Best Practices 100, SEO 100

테스트 결과

Node test runner 총 26개 테스트 통과
캘린더 날짜 유틸 timezone 날짜 밀림 방지 검증
페이지네이션 및 사용자 정보 payload 정규화 검증
쪽지 미읽음 수 정규화와 중복 대화 제거 검증
Playwright 핵심 사용자 흐름 E2E 6개 통과
axe-core 주요 시나리오 접근성 위반 0개
GitHub Actions에서 lint, test, build, E2E 자동 검증

Retrospective

화면 구현을 넘어 사용자 문제를 줄이는 프론트엔드 개발을 경험했습니다.

배운 점

이 프로젝트에서 가장 크게 배운 것은 화면 구현과 사용자 문제 해결은 다르다는 점입니다. ScholarMate는 장학금 정보를 보여주는 데서 끝나는 서비스가 아니라, 사용자가 자신의 조건을 입력하고, 추천을 받고, 관심 장학금을 저장하고, 마감일을 관리하고, 다른 사용자와 정보를 공유하는 흐름을 제공합니다.

저는 이 흐름이 끊기지 않도록 화면, API, 상태 관리, 예외 처리, 성능을 함께 고려했습니다. 또한, 성능 개선 과정에서는 감으로 수정하지 않고 production build, Chrome Performance API, Lighthouse, axe-core로 현재 상태를 측정하고 개선 후 다시 검증했습니다.

개선점

신입 개발자로서 아직 개선할 부분도 분명히 있습니다. 초기에는 기능 구현을 우선해 일부 화면에서 직접 axios와 로컬 상태로 서버 데이터를 관리했습니다.

이후 장학금 목록, 추천, 관심목록, 공지사항처럼 캐싱과 동기화 효과가 큰 화면부터 TanStack Query로 전환했지만, 프로필·캘린더·쪽지처럼 일부 서버 상태 관리 로직은 아직 직접 axios 기반으로 남아 있습니다.

또한, TypeScript를 적용하지 못해 API 응답 구조를 타입으로 보장하지 못한 점도 아쉬운 부분입니다. 향후에는 API 응답 타입을 명확히 정의하고, 남아 있는 서버 상태도 TanStack Query 기반으로 통합해 데이터 흐름을 더 일관되게 개선하고 싶습니다.

ScholarMate 프로젝트 한줄평

사용자 흐름을 연결하고,
성능 병목을 측정 기반으로
개선한 프로젝트

ScholarMate

사용자 맞춤형 장학금 추천 플랫폼

저는 이 프로젝트를 통해 사용자의 문제를 이해하고, 기능을 연결하고, 성능과 안정성을 검증하는 프론트엔드 개발 과정을 경험했습니다.

Thank you

박해준 | Frontend Developer